Artificial Intelligence & Machine Learning (24CSE0316)

Semester-V (Batch-2024)

CREDIT CARD FRAUD DETECTION

Supervised By:

Dr. Righa Tandon

Professor iBeta

CSE Department,

Chitkara University, Punjab

Submitted By:

Anmol Puri - 2410990927

Ashmit Singh - 2410990953

Garvit Singla - 2410991145

Ayush Chauhan - 2410990961

1. Project Title & Overview

Project Title: Credit Card Fraud Detection: A Machine Learning Approach to Identifying Fraudulent Card Transactions

Overview

A card issuer processes millions of transactions every day and must decide, in the fraction of a second before authorisation, whether each one is genuine. This project treats that decision as a supervised binary classification problem: given the attributes of a single transaction, predict whether it is fraudulent.

Continuous Evaluation 1 covers the complete data layer of that project , i.e. collection, cleaning, missing value treatment, feature engineering, categorical encoding, exploratory analysis, feature selection, scaling and the train test split. No model is trained at this stage. The deliverable is an analysis ready dataset, together with the reasoning behind every transformation applied to it.

Dataset Real card transactions made by European cardholders over two days in September 2013, published by the Machine Learning Group at Université Libre de Bruxelles (ULB) and distributed through Kaggle. Twenty eight of the thirty features are principal components released in place of the original attributes to protect cardholder privacy.

Tools. Google Colab, NumPy, Pandas, Matplotlib, Seaborn and Scikit-Learn.

Outcome of this stage A cleaned dataset of 283,726 transactions split into X (35 features) and y (Class), robust-scaled and stratified 80 : 20, ready for model development in CE-2.

Summary of results

Metric	Value
Raw dataset	284,807 rows × 31 columns, ~144 MB CSV
After cleaning	283,726 rows (1,081 exact duplicates removed)
Fraud cases	473 of 283,726 -> 0.167%
Class imbalance	598.8 : 1 (genuine : fraud)
Explicit missing values	0
Disguised missing values investigated	1,808 zero-amount transactions -> retained
Features engineered	5 (HOUR, IS_NIGHT, TIME_PERIOD, LOG_AMOUNT, AMOUNT_BUCKET)
Final feature matrix X	283,726 × 35
Train / test split	226,980 (378 frauds) / 56,746 (95 frauds), stratified

2. Problem Statement

The real world problem. Credit card fraud charges a customer for a purchase they never made. Detection must be automatic and fast, because a human cannot review a stream of millions of transactions, and it must happen before authorisation rather than after the money has moved. In this dataset only 492 of 284,807 transactions are fraudulent is 0.167% of all activity.

Why this is hard Fraud detection is a needle in a haystack problem: genuine transactions outnumber fraudulent ones by roughly 599 to 1. Fixed rules such as "block any transaction above €1,000" fail, because fraudsters deliberately keep amounts small to avoid triggering them. The median fraudulent amount in this dataset is €9.82 against €22.00 for genuine transactions, so an amount threshold would miss the largest cluster of fraud entirely.

How AI/ML addresses it A supervised classifier learns P(fraud | transaction attributes) from 284,807 labelled records and outputs a probability rather than a hard verdict. The issuer can then set its own operating threshold and trade false alarms against missed fraud according to its own cost structure, instead of relying on a single fixed rule. The model also captures interactions between many attributes at once something rule based systems cannot express.

The cost asymmetry A missed fraud costs the issuer the full transaction value plus investigation and reputational cost. A false alarm costs one inconvenienced customer. The two errors are not equally bad, which is why the model must be tuned for recall on the fraud class rather than for overall accuracy.

Why the data must be prepared first The raw file is not model ready. It contains 1,081 exact duplicate transactions, an extreme 598.8 : 1 class imbalance, an Amount column spanning €0 to €25,691 on a completely different scale from the PCA components, and a Time column recorded in raw elapsed seconds that carries no usable signal until it is reshaped. CE-1 addresses all of these.

3. Objectives

1. **To collect and profile** the ULB credit-card transaction dataset (284,807 × 31), documenting its source, file format, data types and the meaning of every column, including the reason V1…V28 are anonymised.
2. **To clean the data** by identifying and removing the 1,081 exact duplicate transactions and validating the ranges of Time, Amount and Class, without discarding legitimate extreme values.
3. **To identify, treat and verify missing values** both explicit nulls and disguised placeholders such as zero-value transactions and prove the dataset is complete after treatment.
4. **To engineer and encode features** by converting Time into an hour of day cycle, log-transforming the skewed Amount, and deriving categorical features that are then label- and one-hot-encoded.
5. **To analyse, select, scale and split** perform exploratory analysis with supporting visualisations, rank features by correlation and mutual information, apply robust scaling without data leakage, and produce a stratified 80 : 20 split defining X and y for CE 2.

4. Dataset Collection & Description

Attribute	Detail
Source	Machine Learning Group, Université Libre de Bruxelles (ULB), via Kaggle (mlg-ulb/credit card fraud)
File name / format	creditcard.csv a single flat CSV, ~144 MB on disk, 67.4 MB in memory
Records	284,807 transactions
Columns	31 Time, V1…V28, Amount, Class
Period and region	Two consecutive days, September 2013, European cardholders
Data types	30 float64 + 1 int64 the file is entirely numeric
Target variable	Class 1 = fraudulent, 0 = genuine
Positive class	492 frauds (0.172% of the raw file)

4.1 Column descriptions

Column	Type	Description
Time	float64	Seconds elapsed since the first transaction in the file (0 – 172,792 s = 48.0 hours). It is a relative counter, not a wall-clock timestamp, and is unusable in its raw form.
V1 … V28	float64	Principal components obtained by applying PCA to the original transaction attributes. The raw fields (merchant, location, device, card identifiers) were never published because they identify real cardholders.
Amount	float64	Transaction value in EUR, ranging €0.00 – €25,691.16, mean €88.47, median €22.00. The only original, unscaled and directly interpretable feature.
Class	int64	The label. 1 = fraudulent transaction, 0 = genuine.

Consequences of the PCA anonymisation. Because V1…V28 are principal components, they are already centred, on a broadly comparable scale, and mutually orthogonal the largest pairwise correlation between any two of them is $|r| = 0.019$. This means there is effectively no multicollinearity to remove, which is unusual for a real dataset. It also means those columns cannot be interpreted individually or combined into meaningful new features, so all feature engineering must target Time and Amount.

Raw Dataset — creditcard.csv (first 5 of 284,807 rows, 10 of 31 columns)

Time	V1	V2	V3	V4	V14	V17	V28	Amount	Class
0.0	-1.36	-0.073	2.536	1.378	-0.311	0.208	-0.021	149.62	0
0.0	1.192	0.266	0.166	0.448	-0.144	-0.115	0.015	2.69	0
1.0	-1.358	-1.34	1.773	0.38	-0.166	1.11	-0.06	378.66	0
1.0	-0.966	-0.185	1.793	-0.863	-0.288	-0.684	0.061	123.5	0
2.0	-1.158	0.878	1.549	0.403	-1.12	-0.237	0.215	69.99	0

Figure 1: First five rows of the raw dataset (10 of 31 columns shown).

5. Data Cleaning

Four checks were carried out, in this order: duplicate records, irrelevant columns, invalid values, and inconsistent data types.

5.1 Duplicate records

df.duplicated().sum() reported 1,081 exact duplicate rows, 19 of which were labelled as fraud. Two transactions sharing an identical 30 dimensional PCA vector and an identical amount cannot be genuinely distinct events, so these are data collection artefacts rather than real repeated purchases. All 1,081 were removed, reducing the dataset from 284,807 to 283,726 rows and the fraud count from 492 to 473.

Why duplicates matter more than usual here: With only 492 fraud examples in total, leaving 19 duplicated frauds in place would allow the same record to appear in both the training set and the test set after splitting. That is direct data leakage: the model would be tested on rows it had already memorised, producing an inflated score that would not survive contact with real traffic.

5.2 Irrelevant columns

The dataset contains no ID column, no free text column and no constant column, so nothing was dropped at this stage. Time and Amount were later replaced by engineered versions (Section 7), but that is a feature-engineering decision made deliberately, not a cleaning reflex.

5.3 Invalid and inconsistent values

Check	Result	Action
Negative Amount	0 rows	None required no impossible values
Zero Amount	1,808 rows (0.637%)	Investigated in Section 6; retained
Negative Time	0 rows	None required
Class outside {0, 1}	0 rows	None required
Amount range	€0.00 – €25,691.16	Retained in full see note below
Data types	30 float64 + 1 int64	No coercion or string cleaning needed

Outliers were deliberately not removed: Applying the standard 1.5 × IQR rule to Amount would flag a large number of high value transactions for deletion, and a substantial share of them are fraudulent. In fraud detection the extreme tail is precisely where much of the signal lives, so trimming outliers would amount to deleting evidence. Range validation was performed instead: every value was confirmed to be physically possible, and all rows were kept.

Result: 284,807 × 31 ⟶ 283,726 × 31. 1,081 duplicates removed, of which 19 were fraud.

6. Missing Values

6.1 Identify

df.isnull().sum().sum() returned 0, and the maximum null count in any single column is also 0. The .info() output confirms 284,807 non null entries in every one of the 31 columns. A missing-value heatmap was plotted as visual confirmation and is uniformly empty.

6.2 Disguised missing values

A null count of zero does not by itself prove a dataset is complete. Real datasets frequently store “unknown” as a placeholder number rather than as an empty cell, so the numeric columns were examined for such placeholders.

The candidate found was Amount = €0.00, which occurs in 1,808 transactions (0.637% of all rows). These are card-verification and pre authorisation checks a real transaction type in which a merchant validates a card without charging it. Critically, 25 of these zero-amount transactions are labelled as fraud, which confirms they are genuine records rather than empty cells.

Decision: retained, neither imputed nor dropped Imputing them would fabricate a value that never existed and would blur a legitimate transaction category. Dropping them would delete 25 of only 473 fraud examples 5.3% of the entire positive class which is an unacceptable loss in a dataset this imbalanced.

6.3 Missing values created during processing

Feature engineering is the most common source of new nulls and infinities. LOG_AMOUNT is computed with np.log1p(x), which evaluates log(1 + x), rather than np.log(x). Because 1,808 rows have Amount = 0 and log(0) is negative infinity, using np.log would have produced 1,808 infinite values and silently corrupted the column; log1p(0) = 0 is well defined.

6.4 Verify

After every engineering step, .isnull().sum() and np.isfinite().all() were re-run. Both confirm 0 nulls and 0 infinities across all 35 final feature columns, with the row count unchanged at 283,726. Verification is what distinguishes “we applied a treatment” from “we proved nothing is left”.

Stage	Nulls	Infinities	Rows
Raw file	0	0	284,807
After de-duplication	0	0	283,726
After feature engineering	0	0	283,726
Final X_train / X_test	0	0	226,980 / 56,746

7. Feature Engineering

V1…V28 are PCA outputs with no interpretable meaning, so they cannot be sensibly combined, ratioed or aggregated. All feature engineering therefore targets the only two original columns in the file: Time and Amount.

New feature	Formula	What it captures
HOUR	(Time // 3600) % 24	Converts a monotonic elapsed-seconds counter into a repeating daily cycle that a model can generalise from.
IS_NIGHT	1 if HOUR in 0–5, else 0	Compresses the strongest time effect into a single binary flag (0.482% fraud at night vs 0.138% by day).
TIME_PERIOD	cut(HOUR) → Night / Morning / Afternoon / Evening	A nominal version of the daily cycle, used to demonstrate one-hot encoding.
LOG_AMOUNT	np.log1p(Amount)	Removes the heavy right skew of Amount; log1p keeps Amount = 0 defined.
AMOUNT_BUCKET	cut(Amount) into 5 ordered bands	An ordinal view of spend size, used to demonstrate label encoding.

Why this matters Raw Time has essentially no correlation with Class and is unusable as a model input it only ever increases, so a model cannot generalise from it. The same column, reshaped into an hour-of-day cycle, exposes one of the clearest patterns in the dataset: the fraud rate varies by a factor of roughly thirty across the day. The signal existed in the file all along; it was invisible until the column was transformed.

After engineering, raw Time and raw Amount were dropped in favour of their engineered replacements, taking the feature count from 31 columns to 35.

8. Categorical Data Encoding

The raw file contains no categorical columns at all: All 31 attributes are numeric and V1…V28 are already PCA outputs. To apply encoding meaningfully, two categorical features were first derived from Time and Amount in Section 7, and each was then encoded using the technique its measurement scale demands.

Feature	Scale	Technique applied	Justification	Result
AMOUNT_BUCKET	Ordinal, 5 levels	Label Encoding → AMOUNT_BUCKET_ENC (0–4)	The bands €0-10 < 10–50 < 50-200 < 200-1000 < 1000+ have a genuine order, so a single integer column preserves it correctly.	1 column
TIME_PERIOD	Nominal, 4 levels	One-Hot Encoding with drop_first=True → TP_Morning, TP_Afternoon, TP_Evening	Night, Morning, Afternoon and Evening have no natural ordering. Label encoding would falsely imply Night > Evening > Afternoon, and a linear model would treat that ordering as real.	3 columns
IS_NIGHT	Binary	None required	A two-level flag is already its own dummy variable.	1 column
V1 … V28, Class	Continuous / binary	None applied	Encoding a continuous PCA component would destroy the information it carries.	unchanged

Why drop_first=True. Retaining all four period columns would make any one of them a perfect linear combination of the other three, because they always sum to one. This is the dummy variable trap: it makes the design matrix singular and breaks linear and logistic regression. Dropping one level (Night, here) makes it the reference category against which the others are measured.

Result: 31 → 35 feature columns.

9. Exploratory Data Analysis & Visualisation

9.1 Target class distribution

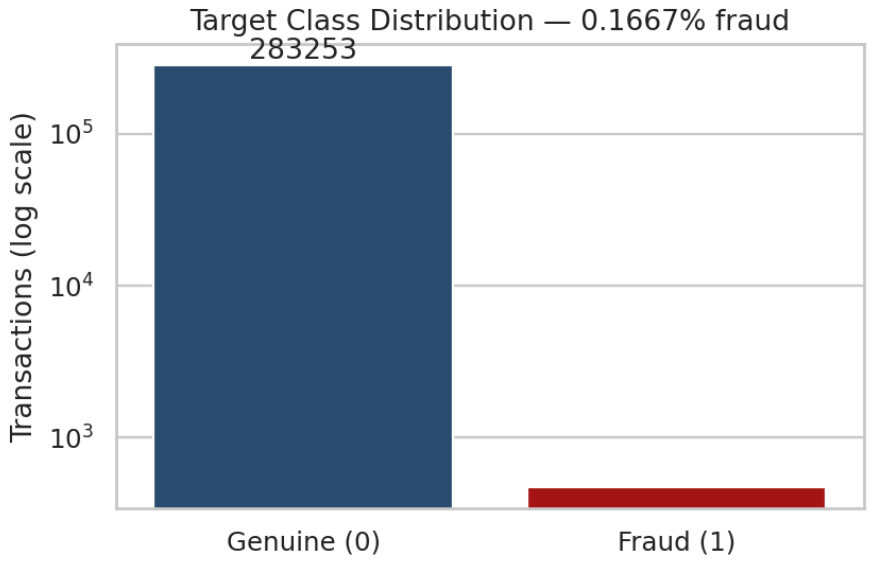

Figure 2: Class distribution on a logarithmic scale.

Observation. After de duplication, 283,253 transactions are genuine and 473 are fraudulent a fraud rate of 0.167% and an imbalance of 598.8 : 1. The vertical axis is logarithmic; on a linear axis the fraud bar is not visible at all.

Implication. Accuracy is a meaningless metric on this dataset. A model that predicts "genuine" for every single transaction achieves 99.83% accuracy while catching zero fraud. Model evaluation in CE-2 must therefore use recall, precision, F1 and the area under the precision recall curve, and every split and cross-validation fold must be stratified.

9.2 Transaction amount distribution

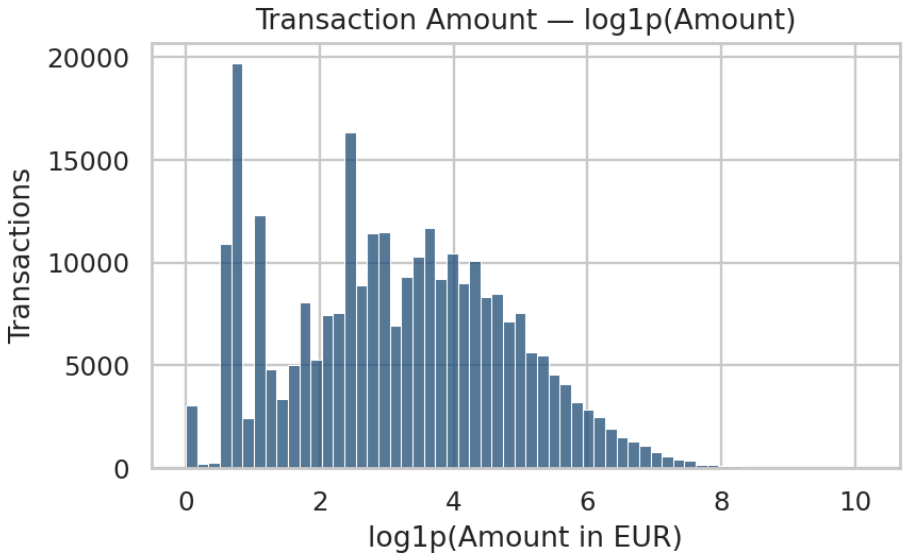

Figure 3: Distribution of log1p(Amount).

Observation. Amount is extremely right skewed: the mean is €88.47 against a median of €22.00, with a maximum of €25,691.16. Plotted on its raw scale the histogram collapses into a single spike near zero and conveys nothing.

Implication. The log1p transform spreads the distribution into a readable shape, and the same transform is used as the model feature. Distance based algorithms such as k-NN and SVM, and gradient-descent optimization generally, both behave poorly on a variable with this degree of skew.

9.3 Fraud rate by hour of day

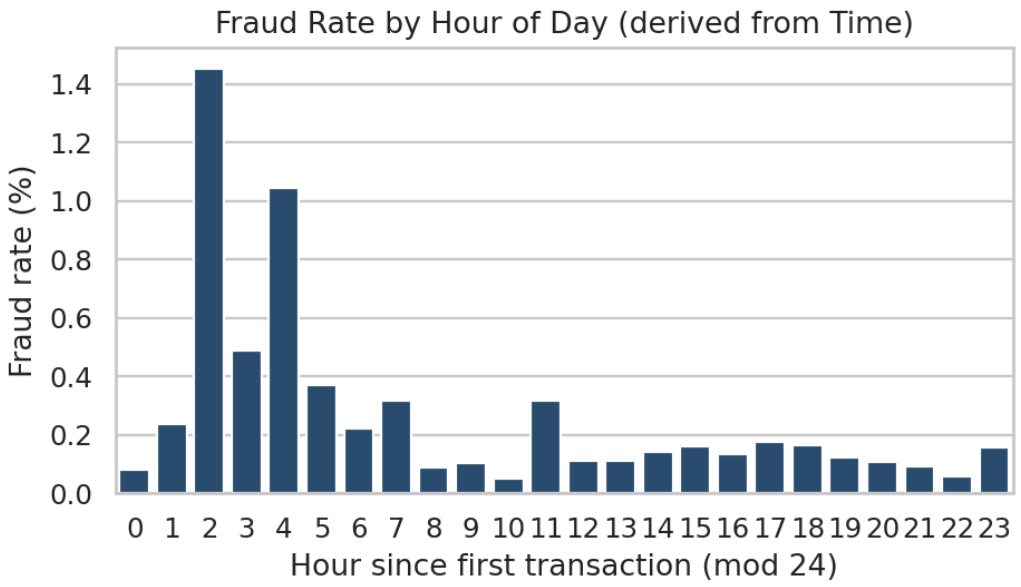

Figure 4: Fraud rate by hour of day, derived from the Time column.

Observation. The fraud rate peaks at hour 2 (1.451%) and hour 4 (1.044%), and reaches its minimum at hour 10 (0.048%) a swing of roughly thirty times across the day. Grouped by period: Night 0.482%, Morning 0.167%, Afternoon 0.138%, Evening 0.115%.

Implication. Night time transactions account for only 8.4% of total volume but carry 3.5 times the fraud rate of daytime transactions. This single chart justifies the entire feature-engineering section: raw Time correlates with nothing, while the same data reshaped into a daily cycle produces one of the strongest interpretable signals available.

9.4 Amount by class

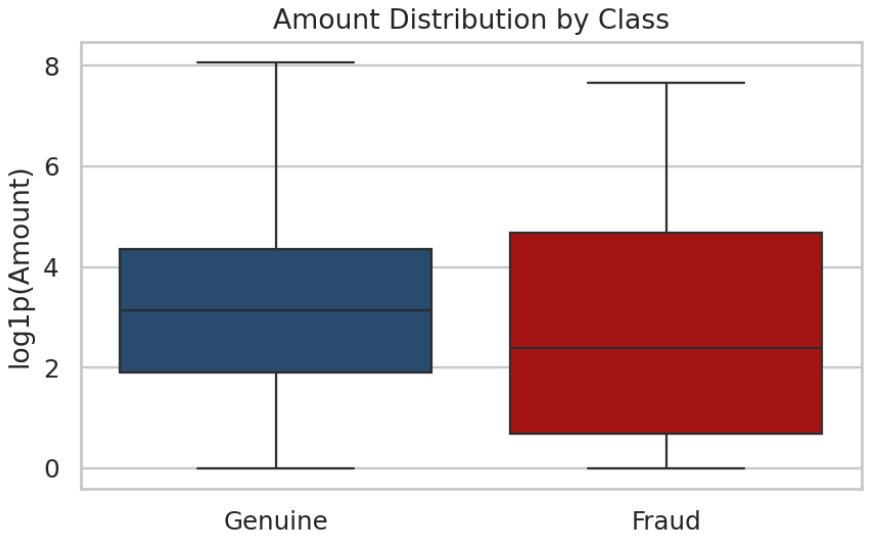

Figure 5: Amount distribution by class (outliers hidden for readability).

Observation. Fraudulent transactions have a higher mean amount (€123.87 against €88.47) but a substantially lower median (€9.82 against €22.00). The fraud rate by amount band is U-shaped: €0 - 10 gives 0.238%, €10 - 50 gives 0.061%, and €1,000+ gives 0.307%.

Implication Fraud amounts are bimodal. There is a large cluster of very small "car testing" transactions, in which a stolen card is probed with a trivial charge to confirm it is live, alongside a smaller number of high value transactions. This is exactly why a single amount threshold fails as a detection rule: it misses the entire low-value cluster, which is the larger of the two.

9.5 Class separation in the PCA components

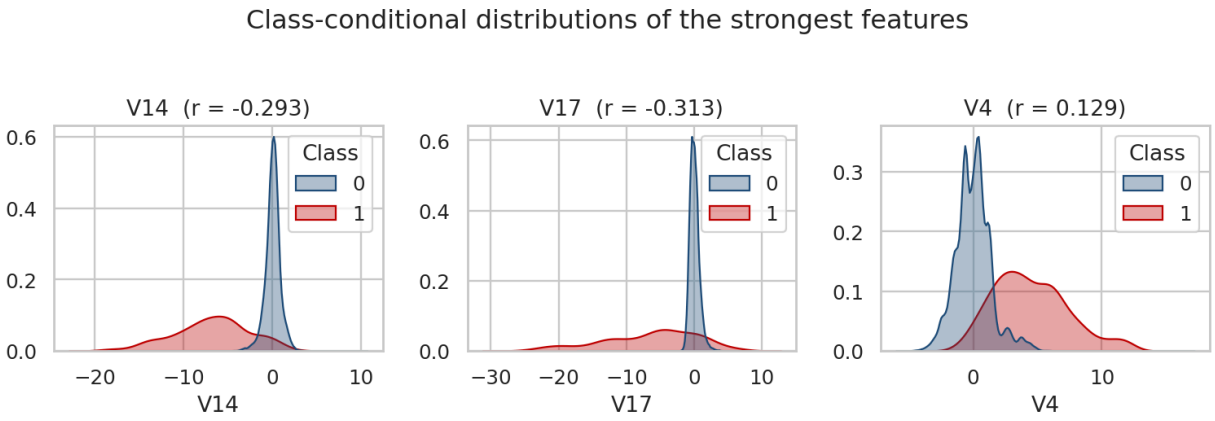

Figure 6: Class-conditional distributions of the strongest features.

Observation. V17 (r = -0.313), V14 (r = -0.293) and V12 (r = -0.251) separate the two classes visibly the fraud density sits well away from the genuine density instead of overlapping it. V4 (r = +0.129) separates them in the opposite direction.

Implication. Although the components are anonymous, this separation demonstrates that they carry real fraud signal. A single threshold on V14 alone would already flag a large share of frauds, which serves as a useful sanity check that the labels and the features are consistent with one another.

9.6 Correlation structure

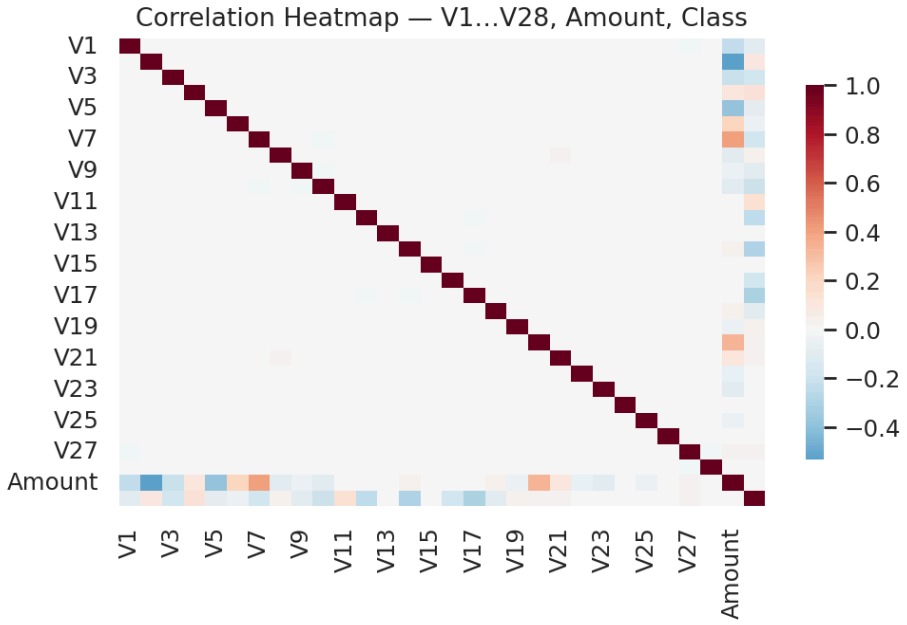

Figure 7: Correlation heatmap of V1…V28, Amount and Class.

Observation. The heatmap is almost entirely neutral away from the diagonal. The largest pairwise correlation between any two of V1…V28 is $|r| = 0.019$.

Implication. PCA produces orthogonal components by construction, so there is effectively no multicollinearity to remove. This is unusual for a real world dataset and is a direct consequence of how the data was anonymised. Correlation-based feature elimination, which would be a routine step on most datasets, is unnecessary here.

10. Feature Selection & Definition of X and Y

Two independent criteria were used, because they measure different things. Pearson correlation captures linear association only, whereas mutual information captures any statistical dependence, including non-linear relationships. Agreement between them is a considerably stronger result than either would be alone.

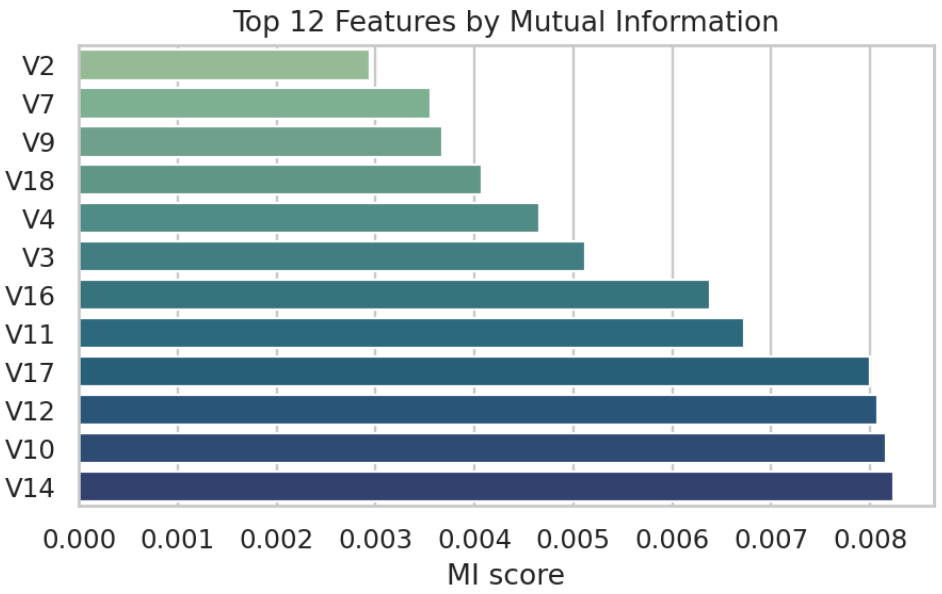

Figure 8: Top 12 features ranked by mutual information with Class.

Observation. Mutual information ranks V14, V10, V12 and V17 highest; Pearson correlation ranks V17, V14, V12 and V10 highest. The same small group of components dominates under both criteria.

10.1 Selection decision

All 35 features were retained. Three reasons support this:

- The PCA components are mutually orthogonal, so no component is redundant with any other. The usual justification for dropping features removing redundancy does not apply.
- With only 473 positive examples, discarding a feature that appears weak on aggregate statistics risks discarding the one signal that captures a rare fraud pattern.
- The only columns actually removed are raw Time and raw Amount, and both were replaced by engineered versions rather than simply discarded.

Aggressive pruning belongs in CE 2, where it can be validated against measured model performance rather than guessed at in advance.

10.2 X and Y

Object	Definition	Shape
X	All 35 engineered and encoded features (V1…V28, HOUR, IS_NIGHT, LOG_AMOUNT, AMOUNT_BUCKET_ENC, TP_Morning, TP_Afternoon, TP_Evening)	283,726 × 35
y	Class the binary fraud label	283,726 × 1

11. Feature Scaling & Train Test Split

11.1 Scaling

Only LOG_AMOUNT and HOUR were scaled. V1…V28 are already PCA outputs on a comparable scale, and the binary flags and one-hot dummies are bounded to {0, 1} by construction.

RobustScaler was used rather than StandardScaler StandardScaler subtracts the mean and divides by the standard deviation, and both statistics are pulled substantially by the €25,691 tail of the Amount distribution. RobustScaler uses the median and the interquartile range instead, neither of which the tail moves. On a fraud dataset where the outliers are the interesting rows, this distinction matters.

Method	Formula	Output range	Appropriate when
Normalisation (MinMaxScaler)	(x − min) / (max − min)	[0, 1]	Features are bounded; neural networks
Standardisation (StandardScaler)	(x − μ) / σ	mean 0, sd 1	Features are roughly Gaussian
Robust scaling (RobustScaler)	(x − median) / IQR	median 0, IQR 1	Heavy outliers are present our case

Preventing data leakage The scaler is fitted on X_train only and then applied to X_test with transform(). Fitting on the full dataset would let the median and IQR of the test set influence the training data, which is data leakage: the resulting test score measures nothing meaningful because the model has already been exposed to information it will not have at prediction time.

11.2 Split

train_test_split(X, y, test_size=0.2, random_state=42, stratify=y) produces:

Set	Shape	Frauds	Fraud rate
X_train	226,980 × 35	378	0.1665%
X_test	56,746 × 35	95	0.1674%

Why stratify=y is essential With 473 positive examples among 283,726 rows, an unstratified random split can easily produce a test set whose fraud rate differs sharply from the training set, making the evaluation unreliable. Stratification forces both sides to carry the same 0.167% proportion. random_state=42 fixes the split so results are reproducible across team members and across runs.

12. Pre Processed Dataset

Final Pre-Processed X_train — 6 of 226,980 rows, 11 of 35 features

row	V14	V17	V12	V10	V4	LOG_AMOUNT	HOUR	IS_NIGHT	AMOUNT_BUCKET_I	TP_Morning	TP_Evening
1	-0.313	0.382	-0.706	1.568	-2.578	0.145	0.111	0	1	0	0
2	0.863	-0.265	1.762	-0.483	-0.038	-0.43	0.778	0	0	0	1
3	-0.689	0.793	1.039	-0.498	0.129	-0.15	-0.444	0	1	1	0
4	1.24	-0.475	-1.775	1.147	3.88	-0.094	-1.556	1	1	0	0
5	-0.045	0.788	0.014	1.493	-1.548	0.032	-0.778	0	1	1	0
6	-2.752	-0.516	0.087	7.406	-1.247	-0.339	0.111	0	0	0	0

Figure 9: First six rows of the final pre-processed X_train (11 of 35 features shown).

Corresponding target vector y_train

row	y_train (Class)
1	0
2	0
3	0
4	0
5	0
6	0

Figure 10: The corresponding target vector y_train.

Verification of the final state:

- 283,726 rows, split into X (35 features) and y (Class).
- 0 duplicate rows, 0 null values and 0 infinities across every column.
- Every column is numeric no object dtype remains, so the frame can be passed directly to any scikit-learn estimator.
- LOG_AMOUNT and HOUR are robust-scaled to median 0 and IQR 1. Negative values in the table above simply indicate below-median transactions; they are not errors.
- Stratified 80 : 20 split: X_train 226,980 × 35 with 378 frauds, X_test 56,746 × 35 with 95 frauds.

13. Conclusion

The raw 284,807 × 31 transaction file has been converted into a clean, fully numeric, analysis ready dataset of 283,726 rows split into X (35 features) and y (Class). Specifically:

- Cleaning removed 1,081 exact duplicate transactions, including 19 duplicated frauds that would otherwise have leaked across the train test boundary and inflated the reported test score.
- Missing values were verified as absent. The 1,808 zero-amount transactions were investigated as possible disguised nulls and deliberately retained, preserving 25 fraud examples that represent 5.3% of the entire positive class.
- Because the file contains no native categorical columns, two were engineered from Time and Amount and encoded appropriately label encoding for the ordinal amount band, one hot encoding for the nominal time period, with drop_first=True to avoid the dummy variable trap.
- Exploratory analysis established that fraud peaks at night (0.482% against 0.138% by day), that fraud amounts are bimodal (median €9.82 against €22.00), and that V17, V14 and V12 separate the classes most cleanly, while the PCA components carry essentially no mutual correlation.
- Five features were engineered from the only two interpretable columns in the file, turning a raw elapsed seconds counter that correlated with nothing into a daily cycle with a 3.5× night-to-day fraud gradient.
- The dataset is robust-scaled without leakage and stratified-split, and is fully ready for model development in CE 2.

14. Project Planning & Future Scope

1. **Baseline models** Train Logistic Regression, Decision Tree and k Nearest Neighbours on the prepared split, recording precision, recall, F1 and PR AUC. Accuracy will not be reported, since it is meaningless at 0.167% positives.
2. **Handling the 598.8 : 1 imbalance** Compare class_weight='balanced', SMOTE oversampling and random under-sampling. Resampling will be applied inside the cross validation folds only resampling before the split duplicates minority rows across the train/test boundary and is the classic mistake that produces fake 99% scores.
3. **Ensembles and hyper-parameter tuning** Random Forest, XGBoost and LightGBM with stratified k fold cross validation, and decision threshold tuning driven by the precision recall curve rather than the default 0.5 cut off.
4. **Anomaly detection as a contrast** Isolation Forest and a One Class SVM trained only on genuine transactions, to test whether an unsupervised approach catches fraud patterns that are absent from the training labels.
5. **Explainability** SHAP values on the best performing model. A declined transaction has to be justifiable to the customer and to the regulator, so an unexplained score is not deployable in a financial setting.
6. **Deployment** Serialise the final model with joblib and expose it through a Streamlit or Flask endpoint that scores a single transaction, with end-to-end latency measured real fraud scoring must complete in milliseconds.

15. References

- Machine Learning Group, Université Libre de Bruxelles Credit Card Fraud Detection dataset, Kaggle (mlg-ulb/creditcardfraud).
- Dal Pozzolo, A., Caelen, O., Johnson, R. A. and Bontempi, G. Calibrating probability with undersampling for unbalanced classification, IEEE Symposium Series on Computational Intelligence, 2015.
- Pedregosa, F. et al. Scikit-learn: Machine Learning in Python, Journal of Machine Learning Research, 2011.
- Scikit-learn documentation Preprocessing data, Feature selection, and Model selection modules.
- McKinney, W. pandas documentation, and Hunter, J. D. Matplotlib documentation.